

Wybrane elementy grafiki w języku Java

Zbigniew Marszalek

Programowanie grafiki

Nowoczesne programy i strony internetowe wykorzystują graficzny interfejs użytkownika GUI (ang. graphical user interface). Java zawiera bibliotekę klas nazwaną AWT (ang. Abstract Window Toolkit). Pierwsza biblioteka AWT budowała elementy interfejsu użytkownika, wywołując i operując rodzimym zestawem GUI, właściwym dla danej platformy (Windows, Solaris, Macintosh itp.). Przykładowo, jeżeli używamy oryginalnej AWT, aby umieścić okno tekstowe, obsługa tekstu była przekazywana odpowiedniemu oknu danego systemu operacyjnego. Rozwiązanie kryjące się za tak napisanymi programami AWT polegało na określeniu położenia i zachowania elementów interfejsu użytkownika, a Java tworzyła odpowiedniki tych elementów. Taki program, w teorii, mógł pracować na każdej platformie, zachowując wygląd docelowego systemu operacyjnego.

Rozwiązanie oparte na odpowiednikach świetnie działało w przypadku małych aplikacji, ale napotykało na problemy przy pisaniu wysokiej jakości przenośnych bibliotek graficznych, zależnych od właściwych dla konkretnej platformy elementów interfejsu użytkownika. Co gorsze, biblioteka AWT wywoływała różne błędy na różnych platformach.

W roku 1996 Netscape stworzył bibliotekę GUI, nazwaną IFC (ang. Internet Foundation Classes), która wykorzystywała zupełnie inne rozwiązanie. Elementy interfejsu użytkownika, takie jak przyciski, menu itp., będą rysowane za powierzchni pustych okien. Firma Sun współpracująca z Netscape udoskonaliła to rozwiązanie i stworzyła bibliotekę o nazwie Swing często nazywanej zestawem Swing. Obecnie Swing jest oficjalną nazwą zestawu narzędzi, który jest częścią JFC (ang. Java Foundation Classes). Elementy JFC to komponenty Swing, API i API 2D.

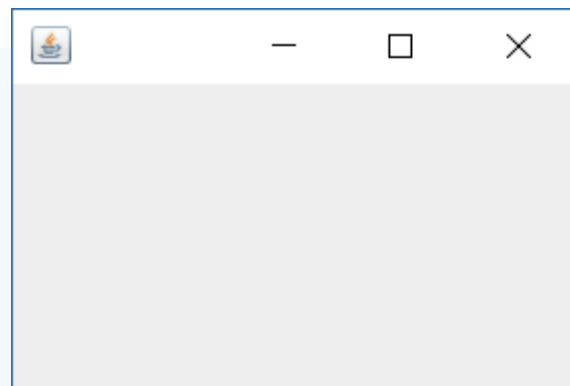
Ramki

Okno najwyższego poziomu (czyli okno, które nie znajduje się wewnątrz innego okna) jest nazywane ramką. Biblioteka AWT posiada klasę *Frame* obsługującą ten najwyższy poziom. Wersja klasy, dostarczana przez Swing, nosi nazwę *JFrame*. *JFrame* jest rozszerzeniem klasy *Frame* i jest jednym z niewielu komponentów klasy Swing, który nie jest rysowany w pustym oknie. Oznacza to, że wszystkie elementy (przyciski, etykiety, pola tekstowe itp.) są rysowane przez menadżera okien systemu operacyjnego, a nie przez Swing. Przedstawiony poniżej program wyświetla na ekranie prostą ramkę.

Tworzenie ramki

```
package c8a;
import javax.swing.*;
public class C8a {
    public static void main(String[] args) {
        ProstaRamka ramka = new ProstaRamka();
        ramka.setDefaultCloseOperation
            (JFrame.EXIT_ON_CLOSE);
        ramka.show();
    }
}
```

```
package c8a;  
import javax.swing.*;  
class ProstaRamka extends JFrame  
{  
    public static final int  
        SZEROKOSC = 300, WYSOKOSC = 200;  
    public ProstaRamka()  
    {  
        setSize(SZEROKOSC, WYSOKOSC);  
    }  
}
```



Pozycjonowanie ramek

Klasa *JFrame* definiuje metody pozwalające zmienić wygląd ramek. Oto niektóre z metod:

- Metoda *dispose* – zamyka okno i usuwa zasoby systemu używane przy jego utworzeniu.
- Metoda *setIconImage* – pobiera obiekt typu *Image* i używa go, gdy okno zostaje wyświetlone.
- Metoda *setTitle* – zmienia tekst w pasku tytułowym okna.
- Metoda *setResizable* – określa, czy użytkownik może zmienić rozmiar okna.

- Metoda *setLocation(x, y)* – umożliwia przesunięcie komponentu. Lewy górny narożnik komponentu zostanie przesunięty do punktu (x,y) względem punktu $(0,0)$, będącego lewym górnym rogiem ekranu.
- Metoda *setBounds(x, y, szerokość, wysokość)* – umożliwia przesunięcie komponentu i zmianę jego rozmiarów.

UWAGA: W przypadku ramki koordynaty dla metod *setLocation* i *setBounds* odnoszą się do całego ekranu.

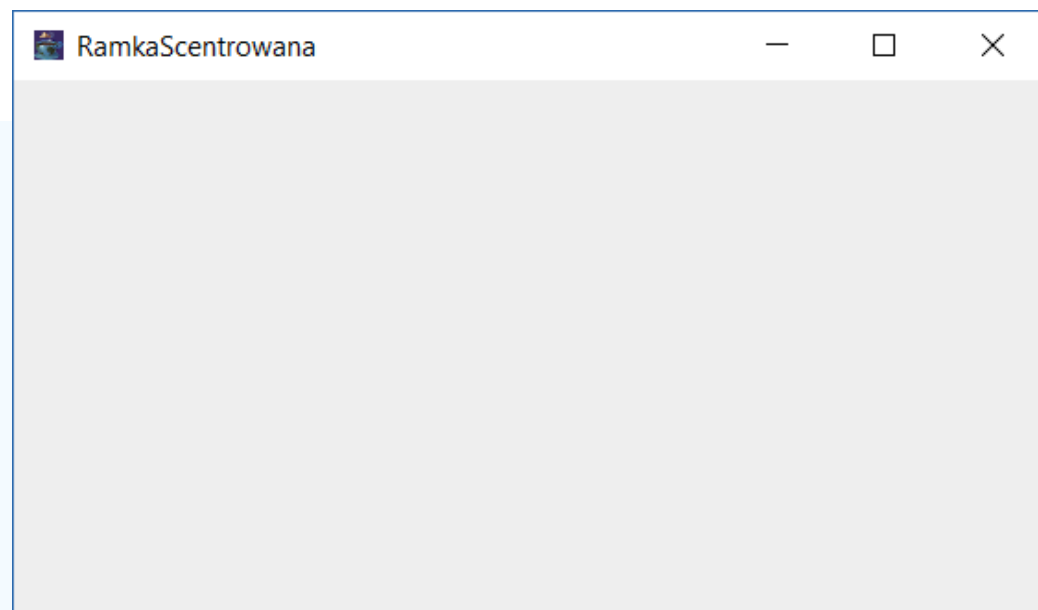
Poniżej przedstawiony program wyświetla ramkę wielkość $\frac{1}{4}$ ekranu wyśrodkowaną względem całego ekranu komputera.

Tworzenie ramki

```
package c8b;
import javax.swing.*;
public class C8b {
    public static void main(String[] args) {
        RamkaScentrowana ramka =
            new RamkaScentrowana();
        ramka.setDefaultCloseOperation
            (JFrame.EXIT_ON_CLOSE);
        ramka.show();
    }
}
```

```
package c8b;
import java.awt.*;
import javax.swing.*;
class RamkaScentrowana extends JFrame{
    public RamkaScentrowana()
    {
        // pobierz rozdzielczość ekranu
        Toolkit zestaw =
            Toolkit.getDefaultToolkit();
        Dimension rozmiarEkranu =
            zestaw.getScreenSize();
        int wysEkranu = rozmiarEkranu.height;
        int szerEkranu = rozmiarEkranu.width;
```

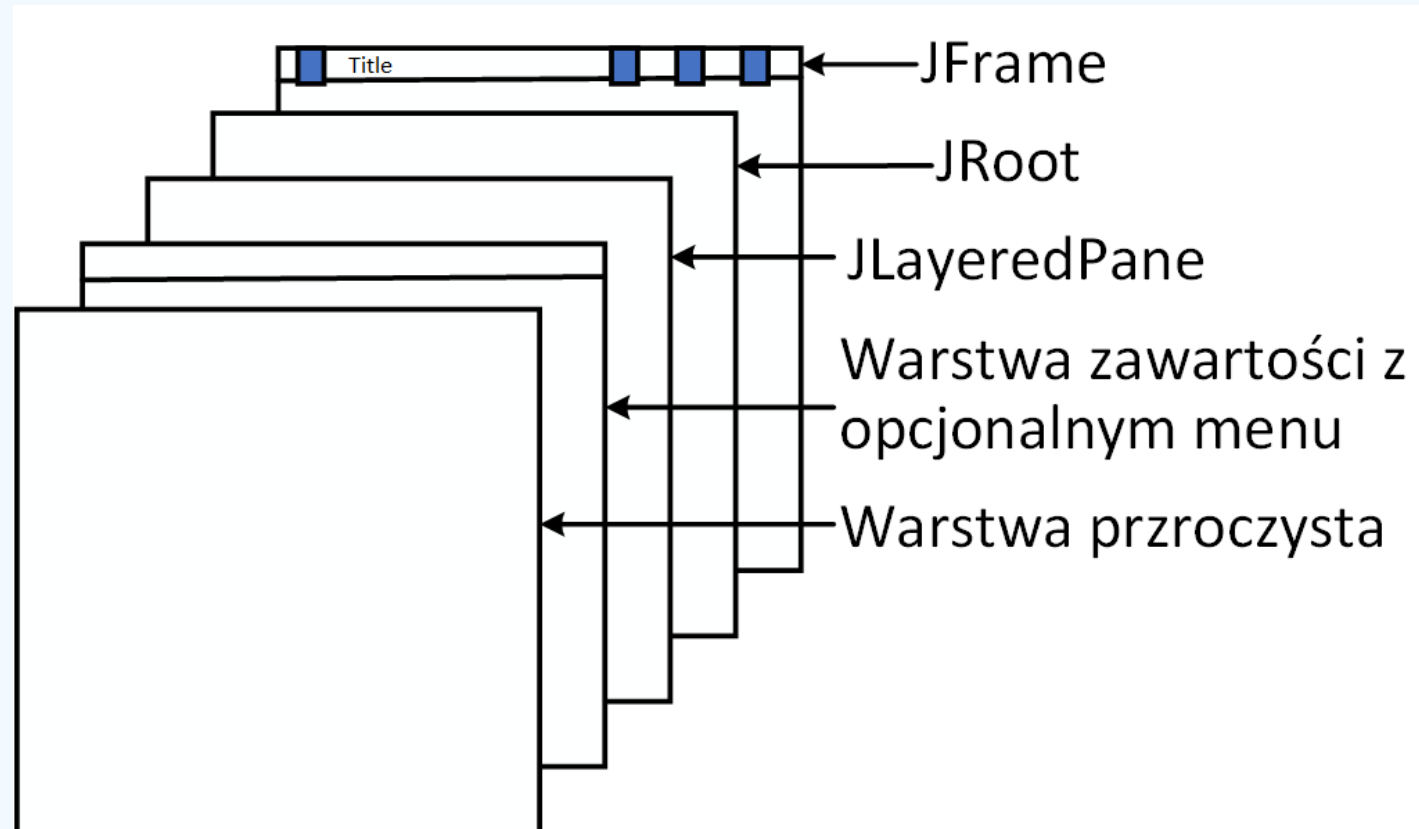
```
// umieść ramkę na środku ekranu
    setSize(szerEkranu / 2, wysEkranu / 2);
    setLocation(szerEkranu / 4, wysEkranu / 4);
// określ ikonę i tytuł ramki
    Image rys = zestaw.getImage("ikona.gif");
    setIconImage(rys);
    setTitle("RamkaScentrowana");
}
}
```



Wyświetlanie informacji w ramce

W języku Java ramki są pojemnikami graficznymi na komponenty, takie jak pasek menu lub inne elementy interfejsu użytkownika.

Rysowania odbywa się na warstwie nazwanej *panel*. Struktura klasy *JFrame* została przedstawiona na rysunku.



W klasie *JFrame* znajdują się cztery warstwy lub okna (ang. panes). Warstwa podstawowa (ang. root pane), warstwa powierzchni (ang. layered pane) oraz powierzchnia przezroczysta (ang. glass pane) są potrzebne do implementacji wyglądu ramki i nie mają znaczenia dla projektanta.

Elementem najważniejszym jest powierzchnia zawartości do której dołączane są komponenty interfejsu użytkownika.

W naszym programie chcemy narysować łańcuch „To nie jest program Witaj, Świecie” w domyślnym oknie, w połowie wysokości i jednej czwartej długości. Ponieważ nie wiemy jak zmierzyć długość łańcuch, rozpoczynamy rysowanie łańcucha w punkcie (75,100). Dokładniej mówiąc, na wysokości 100 pikseli w dół zostanie umieszczona linia bazowa czcionki.

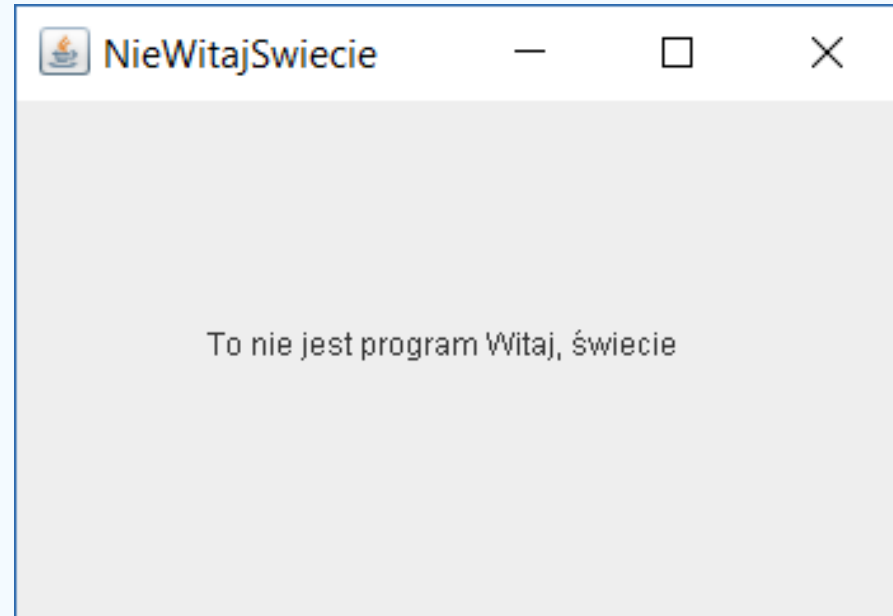
Program może mieć następującą postać:

```
package c8c;  
import javax.swing.*;  
public class C8c {  
    public static void main(String[] args) {  
        RamkaNieWitajSwiecie ramka =  
            new RamkaNieWitajSwiecie();  
        ramka.setDefaultCloseOperation  
            (JFrame.EXIT_ON_CLOSE);  
        ramka.show();  
    }  
}
```

```
package c8c;
import javax.swing.*;
import java.awt.*;
class RamkaNieWitajSwiecie extends JFrame{
    public static final int SZEROKOSC = 300;
    public static final int WYSOKOSC = 200;
    public RamkaNieWitajSwiecie(){
        setTitle("NieWitajSwiecie");
        setSize(SZEROKOSC, WYSOKOSC);
        // dołącz panel do ramki
        PanelNieWitajSwiecie panel =
            new PanelNieWitajSwiecie();
        Container powZawartosci =
            getContentPane();
        powZawartosci.add(panel);
    }
}
```

```
package c8c;
import javax.swing.*;
import java.awt.*;
class PanelNieWitajSwiecie extends JPanel{
    public static final int WIADOMOSC_X = 75;
    public static final int WIADOMOSC_Y = 100;
    @Override
    public void paintComponent(Graphics g){
        super.paintComponent(g);
        g.drawString
            ("To nie jest program Witaj, świecie",
             WIADOMOSC_X, WIADOMOSC_Y);
    }
}
```

Wyświetlanie informacji w ramce



Figury 2D

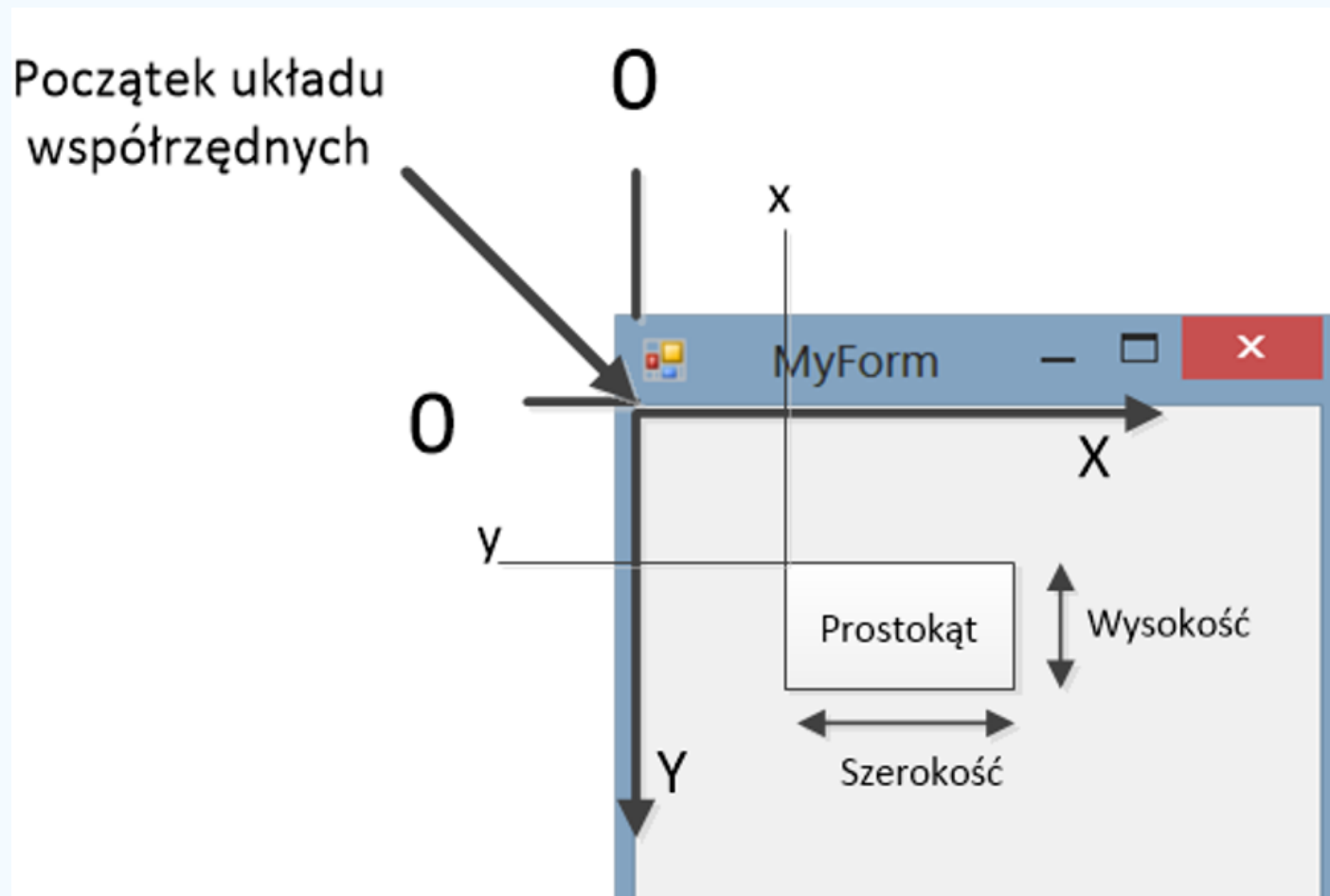
W języku Java dostępna jest klasa *Graphics* posiadająca metody umożliwiające rysowanie figur na warstwie zwartości. Aby rysować figury geometryczne należy utworzyć obiekt klasy *Graphics2D*. Klasa *Graphics2D* jest podklasą klasy *Graphics*. Rysowania figur geometrycznych można wykonać w metodzie *paintComponent*, która domyślnie otrzymuje obiekt klasy *Graphics2D*. Deklaracja przeciążonej metody:

```
@Override
public void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    ...
}
```

Klasa *Rectangle2D* posiada dwie podklasy *Rectangle2D.Float* i *Rectangle2D.Double*, które są statycznymi klasami wewnętrznymi. Gdy tworzymy obiekty typu *Rectangle2D.Float* podajemy koordynaty za pomocą liczb typu *float*, a w przypadku tworzenia obiektów *Rectangle2D.Double* za pomocą liczb typu *double*. Może to wyglądać tak jak poniżej:

```
Rectangle2D.Float floatP =  
    new Rectangle2D.Float  
        (10.0F, 20.0F, 20.0F, 10.0F, 10.0F) ;  
Rectangle2D.Double doubleP =  
    new Rectangle2D.Double  
        (10.0, 20.0, 20.0, 10.0, 10.0) ;
```

Opis elementów grafiki osadzonej w formularzach



Zadanie. Należy napisać program rysujący: prostokąt; elipsę zamkniętą w tym prostokącie; przekątną prostokąta; okrąg ośrodkiem w tym samym punkcie, co środek prostokąta.

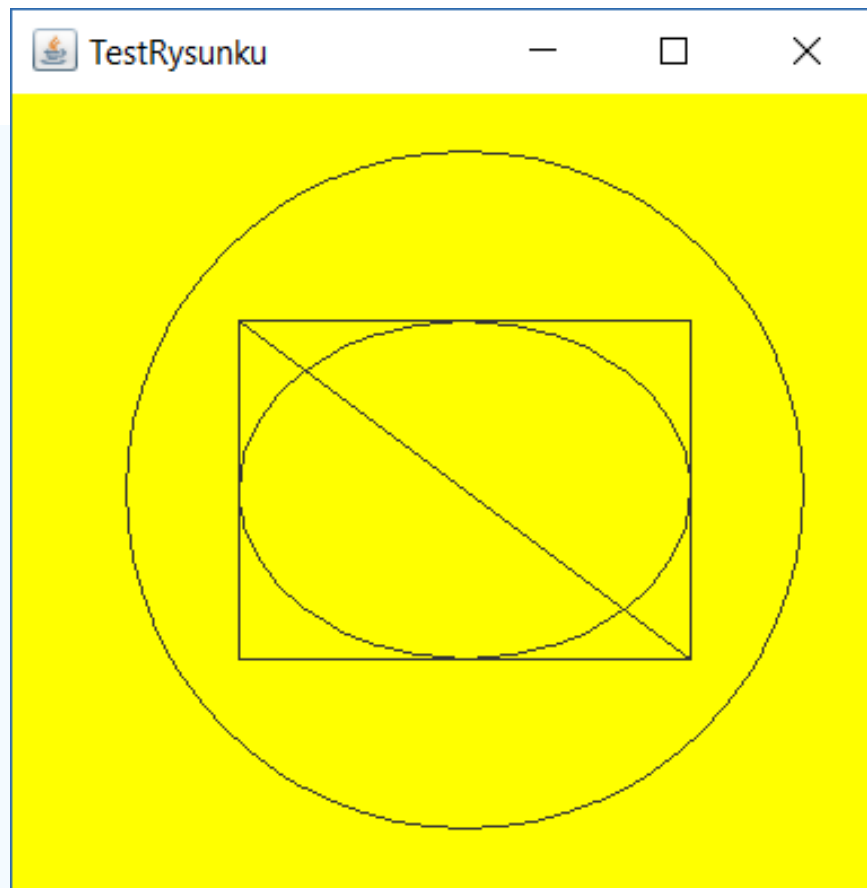
```
package c8d;
import javax.swing.*.*;
public class C8d {
    public static void main(String[] args) {
        RamkaRysunku ramka =
            new RamkaRysunku();
        ramka.setDefaultCloseOperation
            (JFrame.EXIT_ON_CLOSE);
        ramka.setVisible(true);
    }
}
```

```
package c8d;
import java.awt.*;
import javax.swing.*;
class RamkaRysunku extends JFrame{
    public static final int SZEROKOSC = 400;
    public static final int WYSOKOSC = 400;
    public RamkaRysunku() {
        setTitle("TestRysunku");
        setSize(SZEROKOSC, WYSOKOSC);
        // dołącz panel do ramki
        PanelRysunku panel = new PanelRysunku();
        panel.setBackground(Color.YELLOW);
        Container powZawartosci =
            getContentPane();
        powZawartosci.add(panel);
    }
}
```

```
package c8d;
import java.awt.*;
import java.awt.geom.*;
import javax.swing.*;
class PanelRysunku extends JPanel{
    @Override
    public void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        // rysuj prostokąt
        double lewyX = 100;
        double gornyY = 100;
        double szerokosc = 200;
        double wysokosc = 150;
```

```
Rectangle2D pros =  
    new Rectangle2D.Double(lewyX, gornyY,  
        szerokosc, wysokosc);  
g2.draw(pros);  
// rysuj zamkniętą w nim elipsę  
Ellipse2D elipsa =  
    new Ellipse2D.Double();  
elipsa setFrame(pros);  
g2.draw(elipsa);  
// rysuj przekątną  
g2.draw(new Line2D.Double(lewyX, gornyY,  
    lewyX + szerokosc, gornyY + wysokosc));  
// rysuj okrąg o środku w tym samym punkcie  
double srodekX = pros.getCenterX();  
double srodekY = pros.getCenterY();  
double promien = 150;
```

```
Ellipse2D okrag = new Ellipse2D.Double();  
okrag setFrameFromCenter(srodekX, srodekY,  
srodekX + promien, srodekY + promien);  
g2.draw(okrag);  
}  
}
```



Kolory i wypełnienie figur geometrycznych

Klasa *Color* definiuje kolory do tworzenia obiektów graficznych np.

```
Graphics2D g2 = (Graphics2D) g;  
g2.setPaint(Color.red);  
g2.drawString("Uwaga!", 100, 100);
```

Można zdefiniować kolor RGB (czerwony, zielony, niebieski) używając metody *Color* np.

```
g2.setPaint(new Color(0, 255, 128));
```

Kolor tła możemy określić za pomocą metody *setBackground* klasy *Component*, jednej z nadklas *JPanel* np.

```
MojPanel p = new MojPanel();  
p.setBackground(Color.white);  
powZawartosci.add(p);
```

Dodatkowo istnieją predefiniowane stałe w klasie *SystemColor*. Pola składowe tej klasy zawierają zwierają kolory używane przez różne elementy systemu operacyjnego np.

```
ramka.setBackground(SystemColor.window);
```

Zadanie. Należy napisać program rysujący prostokąt wypełniony kolorem czerwonym, a następnie zawartą w nim elipsę matowo zieloną.

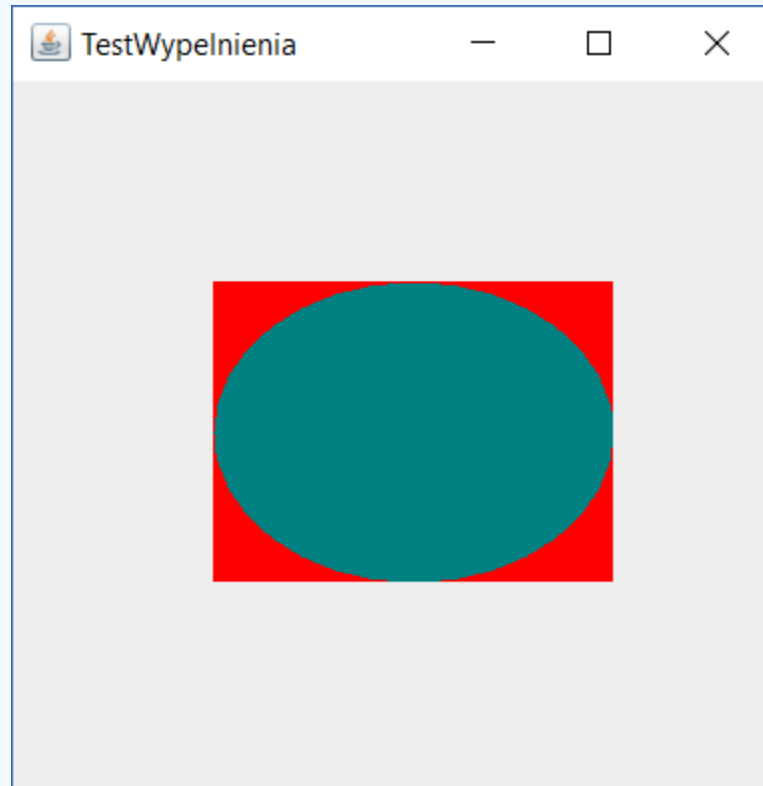
```
package c8e;
import javax.swing.*.*;
public class C8e {
    public static void main(String[] args) {
        FillFrame ramka = new FillFrame();
        ramka.setDefaultCloseOperation
            (JFrame.EXIT_ON_CLOSE);
        ramka.setVisible(true);
    }
}
```

```
package c8e;
import java.awt.*;
import javax.swing.*;
class FillFrame extends JFrame{
    public static final int SZEROKOSC = 400;
    public static final int WYSOKOSC = 400;
    public FillFrame() {
        setTitle("TestWypełnienia");
        setSize(SZEROKOSC, WYSOKOSC);
        // dołącz panel do ramki
        PanelWypełnienia panel =
            new PanelWypełnienia();
        Container powZawartosci = getContentPane();
        powZawartosci.add(panel);
    }
}
```

```
package c8e;
import java.awt.*;
import java.awt.geom.*;
import javax.swing.*;
class PanelWypelnienia extends JPanel{
    @Override
    public void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D)g;
        // narysuj prostokąt
        double lewyX = 100;
        double gornyY = 100;
        double szerokosc = 200;
        double wysokosc = 150;
```

```
Rectangle2D pros = new
    Rectangle2D.Double(lewyX, gornyY,
        szerokosc, wysokosc);
g2.setPaint(Color.red);
g2.fill(pros);
// narysuj zamkniętą w nim elipsę
Ellipse2D elipsa = new Ellipse2D.Double();
elipsa setFrame(pros);
g2.setPaint(new Color(0, 128, 128));
// matowy, niebiesko-zielony kolor
g2.fill(elipsa);
}}
```

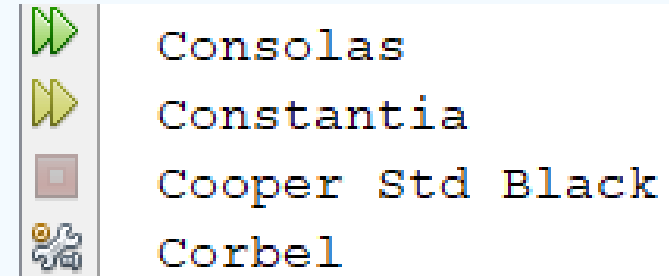
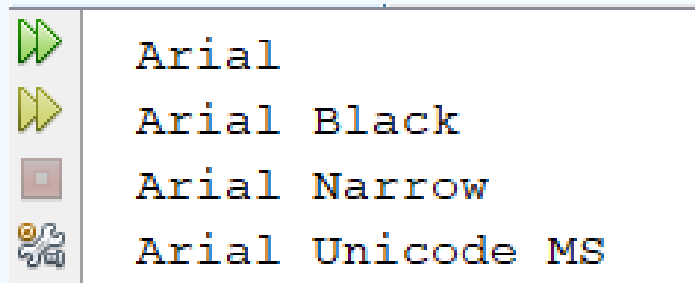
Kolory i wypełnienie figur geometrycznych



Tekst i czcionki

Do wypisania tekstu w oknie możemy używać czcionek dostępnych pod danym systemem operacyjnym w języku Java. Aby dowiedzieć się, jakie czcionki dostępne są komputerze możemy użyć następującego programu:

```
String[] nazwyCzcionek = GraphicsEnvironment  
                        .getLocalGraphicsEnvironment()  
                        .getAvailableFontFamilyNames();  
for(int i = 0; i < nazwyCzcionek.length; i++)  
    System.out.println(nazwyCzcionek[i]);
```



Do wypisania tekstu konstruujemy obiekt klasy Font za pomocą konstruktora klasy Font. Konstruktor przyjmuje trzy argumenty. Pierwszy argument definiuje nazwę czcionki. Drugi określa styl czcionki (normalna, pogrubiona, kursywa lub pogrubiona kursywa). Można używać jednej z następujących stałych jako drugiego parametru:

- Font.PLAIN
- Font.BOLD
- Font.BOLD + Font.ITALIC

Trzecim argumentem jest rozmiar czcionki w punktach.

Przykład utworzenia obiektu Font:

```
Font cz = new Font("Serif", Font.BOLD, 36);
```

Zadanie. Napisać program wyświetlający scentrowany napis „Witaj, świecie!”.

```
package c8g;
import javax.swing.*.*;
public class C8g {
    public static void main(String[] args) {
        RamkaCzcionki ramka = new RamkaCzcionki();
        ramka.setDefaultCloseOperation
            (JFrame.EXIT_ON_CLOSE);
        ramka.setVisible(true);
    }
}
```

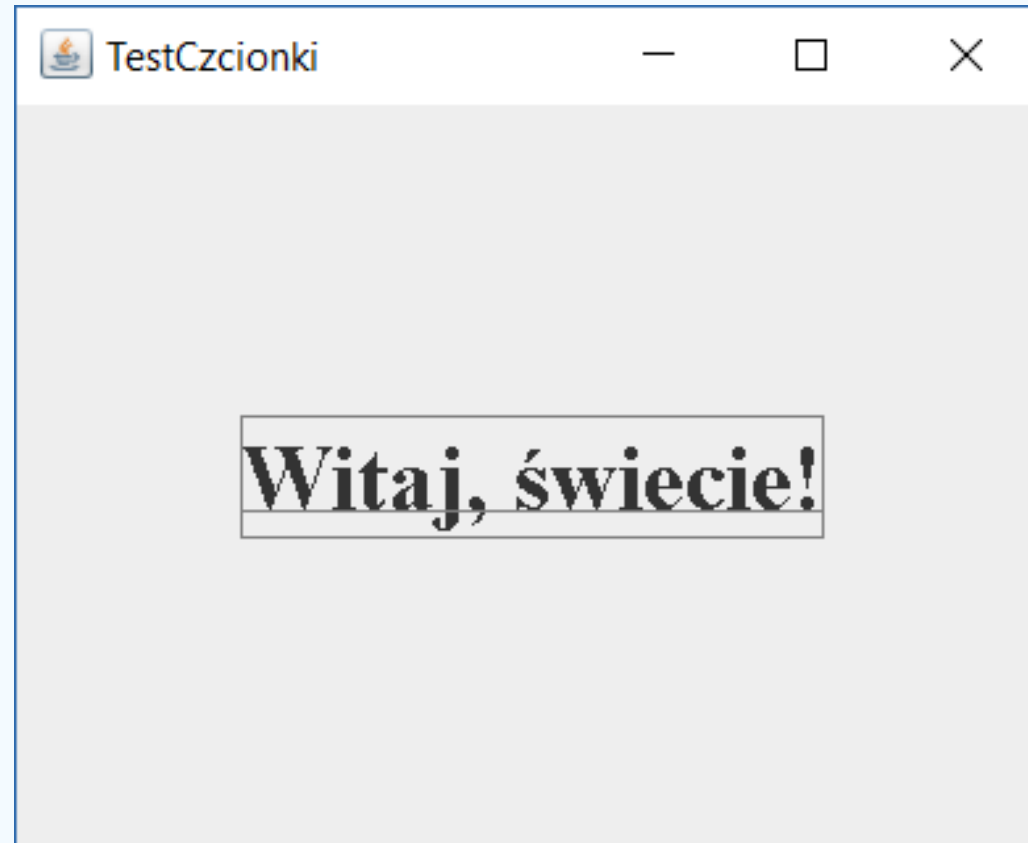
```
package c8g;
import java.awt.*;
import javax.swing.*;
class RamkaCzcionki extends JFrame{
    public static final int SZEROKOSC = 300;
    public static final int WYSOKOSC = 200;
    public RamkaCzcionki(){
        setTitle("TestCzcionki");
        setSize(SZEROKOSC, WYSOKOSC);
// dodaj panel do ramki
        PanelCzcionki panel = new PanelCzcionki();
        Container powZawartosci = getContentPane();
        powZawartosci.add(panel);
    }
}
```

```
package c8g;
import java.awt.*;
import java.awt.font.*;
import java.awt.geom.*;
import javax.swing.*;
class PanelCzcionki extends JPanel{
    @Override
    public void paintComponent(Graphics g){
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D)g;
        String wiadomosc = "Witaj, świecie!";
        Font cz = new Font("Serif",
            Font.BOLD, 36);
        g2.setFont(cz);
    }
}
```

```
// zmierz rozmiar wiadomości
    FontRenderContext kontekst =
        g2.getFontRenderContext();
    Rectangle2D granice = cz.getStringBounds
        (wiadomosc, kontekst);
// (x,y) = lewy górny narożnik tekstu
    double x =
        (getWidth() - granice.getWidth()) / 2;
    double y =
        (getHeight() - granice.getHeight()) / 2;
// dodaj wzniesienie do y,
// aby otrzymać linię bazową
    double wzniesienie = -granice.getY();
    double bazowyY = y + wzniesienie;
```

```
// narysuj wiadomość
    g2.drawString(wiadomosc,
        (int)x, (int)(bazowyY));
    g2.setPaint(Color.gray);
// narysuj linię bazową
    g2.draw(new Line2D.Double(x, bazowyY,
        x + granice.getWidth(), bazowyY));
// narysuj otaczający string prostokąt
    Rectangle2D pros =
        new Rectangle2D.Double(x, y,
            granice.getWidth(),
            granice.getHeight());
    g2.draw(pros);
}}
```

Tekst i czcionki



Obrazy

Za pomocą klasy *Graphics* można wczytywać pliki obrazów i wyświetlać je w oknach projektu. Do wczytania pliku graficznego typu GIF lub JPEG można użyć metody statycznej *getDefaultToolkit* klasy *Toolkit*. Przykładowo w celu pobrania pliku graficznego z domyślnego katalogu projektu wykonanego w NetBeans IDE należy użyć instrukcji:

```
String nazwa = "niebieska-pilka.gif";  
Image obraz = Toolkit.getDefaultToolkit().  
                    getImage(nazwa);
```

W ten sposób zmienna *obraz* zawiera odniesienie do obiektu, który odsługuje dany plik graficzny. Za pomocą metody *drawImage* klasy Graphics można wczytany plik wyświetlić na formularzu.

```
@Override  
public void paintComponent(Graphics g) {  
    ...  
    g.drawImage(obraz, 0, 0, null);  
    ...  
}
```

Przedstawiony kod programu wypełnia całe okno kopiami pliku graficznego. Wypełnienie następuje wewnątrz metody *paintComponent*. Najpierw rysowana jest jedna kopia obrazu w lewym górnym rogu ramki, a następnie za pomocą metody *copyArea* następuje skopiowanie obrazu na cały formularz.

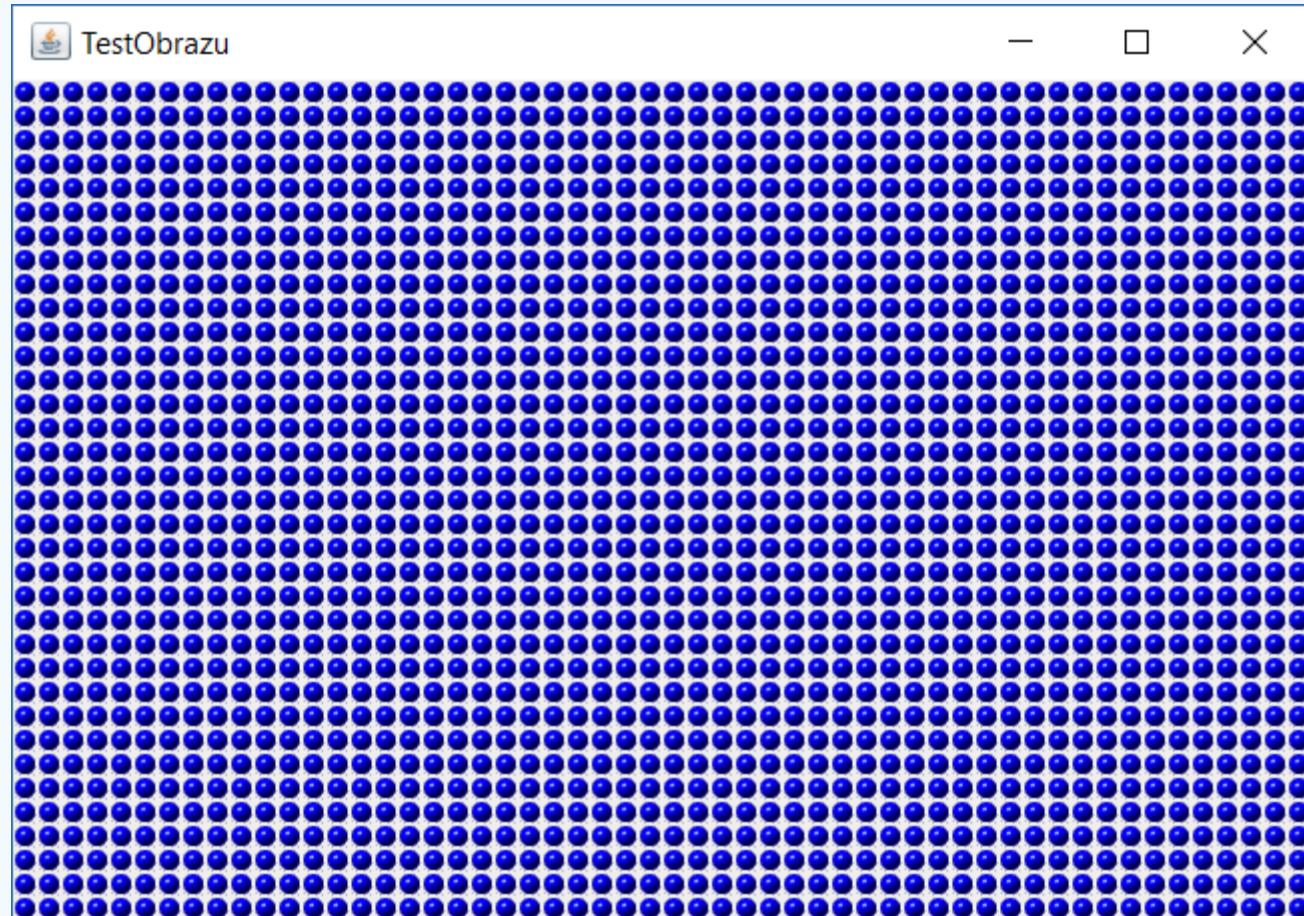
```
package c8h;  
import javax.swing.*;  
public class C8h {  
    public static void main(String[] args) {  
        RamkaObrazu ramka = new RamkaObrazu();  
        ramka.setDefaultCloseOperation  
            (JFrame.EXIT_ON_CLOSE);  
        ramka.setVisible(true);  
    }  
}
```

```
package c8h;
import java.awt.*;
import javax.swing.*;
class RamkaObrazu extends JFrame{
    public static final int SZEROKOSC = 300;
    public static final int WYSOKOSC = 200;
    public RamkaObrazu() {
        setTitle("TestObrazu");
        setSize(SZEROKOSC, WYSOKOSC);
// dodaj panel do ramki
        PanelObrazu panel = new PanelObrazu();
        Container powZawartosci = getContentPane();
        powZawartosci.add(panel);
    }
}
```

```
package c8h;
import java.awt.*;
import javax.swing.*;
class PanelObrazu extends JPanel{
    private Image obraz;
    public PanelObrazu() {
// załaduj obraz
        obraz = Toolkit.getDefaultToolkit().
            getImage("niebieska-pilka.gif");
        MediaTracker trop = new MediaTracker(this);
        trop.addImage(obraz, 0);
        try{ trop.waitForID(0);}
        catch (InterruptedException e) {}
    }
}
```

```
@Override
public void paintComponent(Graphics g) {
    super.paintComponent(g);
    int szerObrazu = obraz.getWidth(this);
    int wysObrazu = obraz.getHeight(this);
    // narysuj obraz w lewym górnym rogu okna
    g.drawImage(obraz, 0, 0, null);
    // wypełnij panel kopiami obrazu
    for (int i = 0;
        i * szerObrazu <= getWidth(); i++)
        for (int j = 0;
            j * wysObrazu <= getHeight(); j++)
            if (i + j > 0)
                g.copyArea(0, 0, szerObrazu, wysObrazu,
                    i * szerObrazu, j * wysObrazu);
}
```

Test obrazu



Obsługa zdarzeń

Obsługa zdarzeń jest priorytetowym zagadnieniem programowania interfejsu użytkownika. Każde środowisko GUI musi monitorować zdarzenia takie jak przyciśnięcie klawisza lub kliknięcie myszką. Środowisko operacyjne informuje o tych zdarzeniach programy, które w danym momencie działają.

Każdy z programów musi zdecydować, w jaki sposób. W różnych językach programowania obsługa zdarzeń został rozwiązana w różny sposób. Java przyjmuje rozwiązanie pośrednie pomiędzy rozwiązaniem Visual Basic a czystym C. W ramach modelu zdarzeń obsługiwanych przez AWT zdarzenia są przesyłane od źródeł zdarzeń do słuchaczy zdarzeń. Ten tzw. Model delegacji zdarzeń jest bardziej elastyczny niż w innych językach programowania.

Źródła zdarzeń posiadają metody, które pozwalają przyporządkować słuchaczy do odpowiednich zdarzeń. Gdy dane zdarzenie zostanie wywołane, źródło wysyła informacje o tym zdarzeniu do wszystkich obiektów słuchaczy, przyporządkowanych temu zdarzeniu. W języku zorientowanym obiektowo Java, informacje o zdarzeniu zawarte są w obiekcie zdarzenia. W Javie wszystkie obiekty zdarzeń pochodzą od rozszerzeń klasy `java.util.EventObject`. Klasa ta zawiera podklasy do obsługi każdego typu zdarzenia, takie jak `ActionEvent` i `WindowEvent`. Schemat obsługi zdarzeń jest następujący:

- Obiekt słuchacza jest instancją klasy, która implementuje interfejs słuchacza.

- Źródło zdarzenia jest obiektem potrafiącym rejestrować obiekty słuchacza i wysyłać im obiekty zdarzeń.
- Źródło zdarzeń wysyła obiekty zdarzeń do wszystkich zarejestrowanych słuchaczy, gdy tylko wystąpi zdarzenie.
- Obiekt słuchacz na podstawie przekazanej informacji określa sposób reakcji na dane zdarzenie.

Przykładowy obiekt słuchacza źródła może wyglądać jak poniżej:

```
ActionLisner sluchacz = ....;  
JButton przycisk = new JButton("OK");  
przycisk.addActionListener(sluchacz);
```

Powyższy kod wymaga, aby klasa, do której należy obiekt słuchacza, implementowała odpowiedni interfejs (w tym wypadku jest to interfejs *ActionListener*). Aby zaimplementować interfejs *ActionListener*, klasa słuchacza musi posiadać metodę o nazwie *actionPerformed*, która jako parametr otrzymuje obiekt typu *ActionEvent*.

```
private class DzialanieKoloru
    implements ActionListener{
    .....
    @Override
    public void actionPerformed
        (ActionEvent zdarzenie){
// Kod reagujący na kliknięcie przycisku
        repaint();
    }
}
```

Przedstawiony poniżej program przedstawia obsługę zdarzenia kliknięcia w jeden z przycisków. Odpowiedni obiekt słuchacza zmienia kolor tła panelu.

```
package c8i;  
import javax.swing.*;  
public class C8i {  
    public static void main(String[] args) {  
        RamkaPrzyciskow ramka = new RamkaPrzyciskow();  
        ramka.setDefaultCloseOperation  
            (JFrame.EXIT_ON_CLOSE);  
        ramka.setVisible(true);  
    }  
}
```

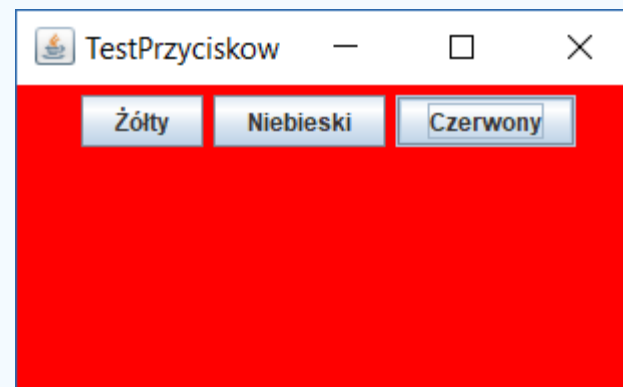
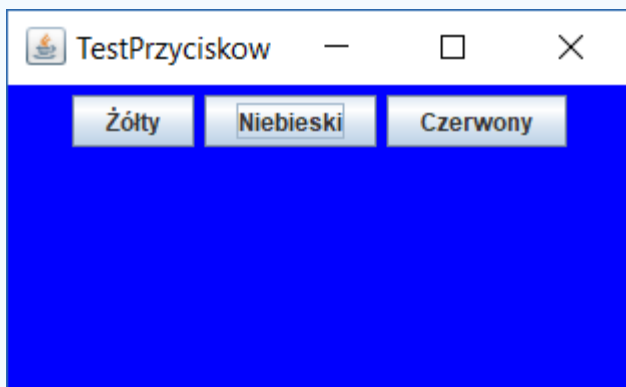
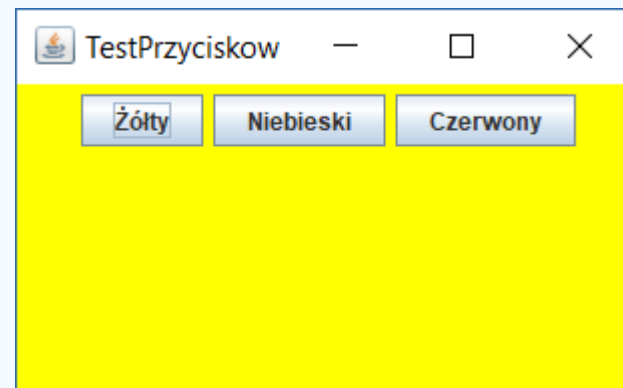
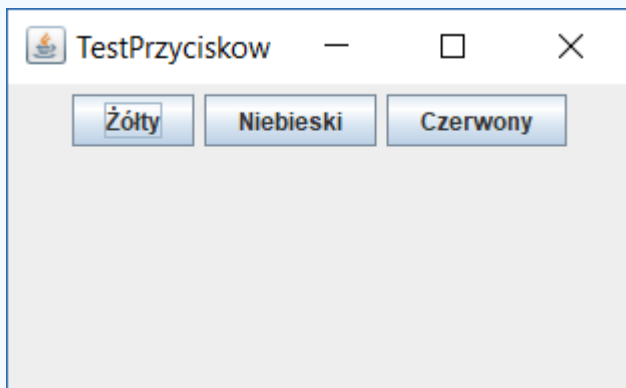
```
package c8i;
import java.awt.*;
import javax.swing.*;
class RamkaPrzyciskow extends JFrame{
    public static final int SZEROKOSC = 300;
    public static final int WYSOKOSC = 200;
    public RamkaPrzyciskow() {
        setTitle("TestPrzyciskow");
        setSize(SZEROKOSC, WYSOKOSC);
        // dołącz panel do ramki
        PanelPrzyciskow panel =
            new PanelPrzyciskow();
        Container powZawartosci =
            getContentPane();
        powZawartosci.add(panel);
    }
}
```

```
package c8i;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class PanelPrzyciskow extends JPanel{
    public PanelPrzyciskow() {
// skonstruuuj przyciski
        JButton przyciskZolty =
            new JButton("Żółty");
        JButton przyciskNiebieski =
            new JButton("Niebieski");
        JButton przyciskCzerwony =
            new JButton("Czerwony");
```

```
// dołącz przyciski do panelu
    add(przyciskZolty);
    add(przyciskNiebieski);
    add(przyciskCzerwony);
// skonstruuuj działania przycisków
    DzialanieKoloru dzialanieZoltego = new
        DzialanieKoloru(Color.yellow);
    DzialanieKoloru dzialanieNiebieskiego =
        new DzialanieKoloru(Color.blue);
    DzialanieKoloru dzialanieCzerwonego =
        new DzialanieKoloru(Color.red);
// przyporządkuj działania do przycisków
    przyciskZolty.addActionListener
        (dzialanieZoltego);
    przyciskNiebieski.addActionListener
        (dzialanieNiebieskiego);
```

```
przyciskCzerwony.addActionListener
    (dzialanieCzerwonego);
}
// Słuchacz, zmieniający kolor tła panelu
private class DzialanieKoloru
    implements ActionListener{
    private Color kolorTla;
    public DzialanieKoloru(Color k) {
        kolorTla = k;
    }
    @Override
    public void actionPerformed
        (ActionEvent zdarzenie) {
        setBackground(kolorTla);
        repaint();
    }
}
```

Obsługa zdarzenia kliknięcia w przycisk



Zdarzenia semantyczne i zdarzenia niskiego poziomu

AWT tworzy podział pomiędzy zdarzeniami niskiego poziomu a zdarzeniami semantycznymi. Zdarzenie semantyczne to takie, które opisuje, co robi użytkownik, czyli użytkownik klika w ten przycisk np. zdarzenie *ActionEvent* jest zdarzeniem semantycznym. Zdarzenia niskiego poziomu to takie, które umożliwiają takie działanie. W przykładzie panelu z przyciskami zdarzeniem niskiego poziomu jest naciśnięcie klawisza myszki, seria poruszeń oraz zwolnienie klawisz myszki (jeżeli zwolnienie klawisza myszki następuje w obszarze przycisku). Analogicznie, zdarzeniem semantycznym jest przesunięcie paska przewijania, ale wykonane w tym samym czasie poruszanie myszką jest zdarzeniem niskiego poziomu.

Aktywacja komponentów

Poruszając myszką, możemy wskazać dowolny obiekt na ekranie monitora. Przykładowo, jeżeli wpisujemy tekst, to informacja trafia do konkretnego obiektu. Menedżer okien przesyła kody klawiszy do aktywnego okna. W danym momencie tylko jedno okno może być aktywne.

Zauważmy teraz, że aktywne okno jest kontrolowane przez program Javy. Okno Javy otrzymuje kod naciskanych klawiszy i przesyła do odpowiedniego komponentu. O tym komponentcie mówimy, że jest aktywny. Tak jak aktywne okno odróżnia się od innych, tak samo większość komponentów Swing zmienia swój wygląd po aktywowaniu. Gdy aktywne jest pole tekstowe, można w nie wpisać dowolny tekst. Gdy, aktywny jest przycisk, można kliknąć w niego, naciskając spację.

Zdarzenia dotyczące klawiatury

Gdy użytkownik wciska klawisz, generowane jest zdarzenie `KEY_PRESSED` *KeyEvent*. Gdy klawisz zostaje zwolniony, następuje `KEY_RELEASE` *KeyEvent*. Zdarzenia te są przechwytywane przez metody *keyPressed* i *keyReleased*, należące do klasy, która implementuje interfejs *KeyListener*. Korzystając z tych metod można przechwycić zdarzenie naciśnięcia klawisza. Metoda *keyTyped* zwraca informacje o znaku wygenerowanym przez naciśnięci danego klawisza.

Opiszmy teraz jakie zdarzenia są generowane przy wpisaniu dużej litery A:

- Został naciśnięty klawisz SHIFT (wywołanie *keyPressed* dla `VK_SHIFT`).

- Został naciśnięty klawisz A (wywołanie *keyPressed* dla VK_A).
- Zostało wysłane „A” (wywołanie *keyTyped* dla „A”).
- Został zwolniony klawisz A (wywołanie *keyReleased* dla VK_A).
- Został zwolniony klawisz SHIFT (wywołanie *keyReleased* dla VK_SHIFT).

Na początku implementacji metod *keyPressed* i *keyReleased* należy sprawdzić kod klawisza. Dla przykładu, poniższy kod sprawdza czy użytkownik nacisnął kombinację SHIFT + PRAWA STRZAŁKA.

Uwaga: Do metody *keyTyped* zostają przesłane kody wyłącznie tych klawiszy, które generują znaki Unicode. Aby sprawdzić stan klawiszy strzałek itp. Należy użyć metody *keyPressed*.

```
public void keyPressed(KeyEvent zdarzenie) {  
    int kodKlawisza = zdarzenie.getKeyCode();  
    if(kodKlawisza == KeyEvent.VK_RIGHT &&  
        zdarzenie.isShiftDown())  
    {  
        .....  
    }  
}
```

Wewnątrz metody *keyTyped* można wywołać metodę *getKeyChar* i pobrać wartość znaku który został przyciśnięty.

Przedstawiony poniżej program przedstawia obsługę naciskania klawiszy h, j, k i l. Program jest uproszczoną implementacją zabawki o nazwie Etch-Asketch.

```
package c8k;
import javax.swing.*.*;
public class C8k {
    public static void main(String[] args) {
        RamkaSzkicu ramka = new RamkaSzkicu();
        ramka.setDefaultCloseOperation
            (JFrame.EXIT_ON_CLOSE);
        ramka.setVisible(true);
    }
}
```

```
package c8k;
import java.awt.*;
import javax.swing.*;
class RamkaSzkicu extends JFrame{
    public static final int SZEROKOSC = 300;
    public static final int WYSOKOSC = 200;
    public RamkaSzkicu() {
        setTitle("Szkic");
        setSize(SZEROKOSC, WYSOKOSC);
        // dołącz panel do ramki
        PanelSzkicu panel = new PanelSzkicu();
        Container powZawartosci = getContentPane();
        powZawartosci.add(panel);
    }
}
```

```
package c8k;
import java.awt.*;
import java.awt.geom.*;
import java.util.*;
import java.awt.event.*;
import javax.swing.*;
class PanelSzkicu extends JPanel{
    private Point2D ostatni;
    private ArrayList linie;
    private static final int
        PREDKOSC_NORMALNA = 1;
    private static final int
        PREDKOSC_ZWIEKSZONA = 5;
```

```
public PanelSzkicu() {
    ostatni = new Point2D.Double(100, 100);
    linie = new ArrayList();
    UchwytyKlawisza sluchacz =
        new UchwytyKlawisza();
    addKeyListener(sluchacz);
}
@Override
public boolean isFocusTraversable() {
    return true;
}
// zezwala na aktywacje panelu
}
//Dodaje nową linię do szkicu
//@param dx przesunięcie poziome
//@param dy przesunięcie pionowe
```

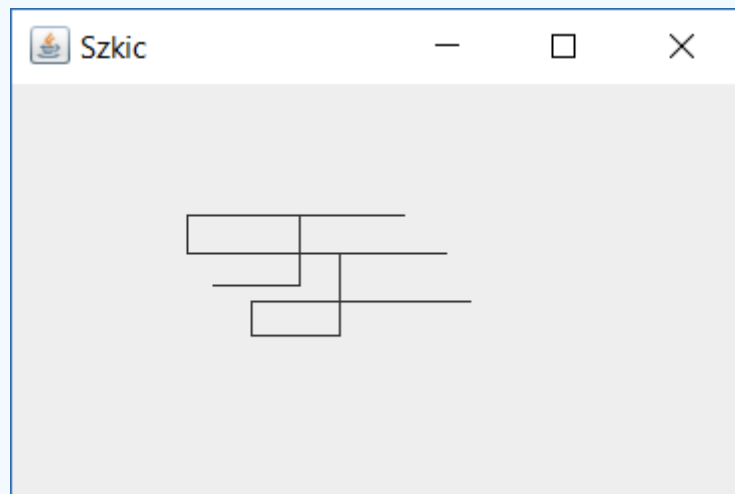
```
    public void add(int dx, int dy) {  
        // wyznacz nowy punkt końcowy  
        Point2D koniec = new Point2D.Double  
            (ostatni.getX() + dx,  
             ostatni.getY() + dy);  
        // dołącz nową linię  
        Line2D linia = new Line2D.Double  
            (ostatni, koniec);  
        linie.add(linia);  
        repaint();  
        // zapamiętaj nowy punkt końcowy  
        ostatni = koniec;  
    }
```

```
@Override
public void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D)g;
    // narysuj wszystkie linie
    for (int i = 0; i < linie.size(); i++)
        g2.draw((Line2D)linie.get(i));
}
```

```
@Override
public void keyPressed(KeyEvent zdarzenie) {
    int kodKlawisza = zdarzenie.getKeyCode();
    // wyznacz dystans
    int d;
    if (zdarzenie.isShiftDown())
        d = PREDKOSC_ZWIEKSZONA;
    else
        d = PREDKOSC_NORMALNA;
    // dołącz nową linię
    if (kodKlawisza == KeyEvent.VK_LEFT) add(-d, 0);
    else if (kodKlawisza == KeyEvent.VK_RIGHT) add(d, 0);
    else if (kodKlawisza == KeyEvent.VK_UP) add(0, -d);
    else if (kodKlawisza == KeyEvent.VK_DOWN) add(0, d);
}
```

```
@Override
public void keyReleased(KeyEvent zdarzenie) {}
@Override
public void keyTyped(KeyEvent zdarzenie) {
    char znakKlawisza = zdarzenie.getKeyChar();
// wyznacz dystans
    int d;
    if(Character.isUpperCase(znakKlawisza)) {
        d = PREDKOSC_ZWIEKSZONA;
        znakKlawisza = Character.toLowerCase
                                   (znakKlawisza);
    }
    else
        d = PREDKOSC_NORMALNA;
// dołącz nową linię
```

```
    if (znakKlawisza == 'h') add(-d, 0);  
    else if (znakKlawisza == 'l') add(d, 0);  
    else if (znakKlawisza == 'k') add(0, -d);  
    else if (znakKlawisza == 'j') add(0, d);  
}  
}  
}
```



Zdarzenia dotyczące myszki

Za pomocą myszki można rysować przy pomocy przechwytywania zdarzeń: poruszania myszki, kliknięcia i przesuwania myszki przy wciśniętym klawiszu. Gdy użytkownik naciska klawisz myszki, wywołuje trzy metody słuchacza: kolejno *mousePressed* (gdy klawisz zostaje wciśnięty), *mouseReleased* (gdy klawisz zostaje zwolniony) i w końcu *mouseClicked*.

W przedstawionym programie wykorzystano dwa typy zdarzeń dotyczące myszki. Zdefiniowane zostały dwie klasy wewnętrzne: *UchwytMyszki* oraz *UchwytRuchuMyszki*. Klasa *UchwytMyszki* rozszerza klasę *MouseListener* i definiuje tylko dwie z pięciu metod interfejsu *MouseListener*. Klasa *UchwytRuchuMyszki* implementuje *MouseMotionListener* i definiuje obydwie metody tego interfejsu.

```
package c8j;  
import javax.swing.*;  
public class C8j {  
    public static void main(String[] args) {  
        RamkaMyszki ramka = new RamkaMyszki();  
        ramka.setDefaultCloseOperation  
            (JFrame.EXIT_ON_CLOSE);  
        ramka.setVisible(true);  
    }  
}
```

```
package c8j;  
import java.awt.*;  
import javax.swing.*;  
class RamkaMyszki extends JFrame{  
    public static final int  
        SZEROKOSC = 300, WYSOKOSC = 200;  
    public RamkaMyszki(){  
        setTitle("TestMyszki");  
        setSize(300, 200);  
        // dołącz panel do ramki  
        PanelMyszki panel =  
            new PanelMyszki();  
        Container powZawartosci =  
            getContentPane();  
        powZawartosci.add(panel);  
    }  
}
```

```
package c8j;  
  
import java.awt.*;  
import java.awt.event.*;  
import java.util.*;  
import java.awt.geom.*;  
import javax.swing.*;  
  
class PanelMyszki extends JPanel{  
    public PanelMyszki(){  
        kwadraty = new ArrayList();  
        obecny = null;  
        addMouseListener  
            (new UchwytMyszki());  
        addMouseMotionListener  
            (new UchwytRuchuMyszki());  
    }  
}
```

```
@Override
public void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    // narysuj wszystkie kwadraty
    for (int i = 0; i < kwadraty.size(); i++)
        g2.draw((Rectangle2D) kwadraty.get(i));
}
//Znajdź pierwszy kwadrat, zawierający dany punkt
//@param p punkt
//@return indeks pierwszego kwadratu
```

```
public Rectangle2D find(Point2D p) {  
    for (int i = 0; i < kwadraty.size(); i++) {  
        Rectangle2D r =  
            (Rectangle2D) kwadraty.get(i);  
        if (r.contains(p)) return r;  
    }  
    return null;  
}  
  
//Dołącza nowy kwadrat do zbioru  
//@param p środek kwadratu  
public void add(Point2D p) {  
    double x = p.getX();  
    double y = p.getY();
```

```
        obecnny = new Rectangle2D.Double (
            x - DLUGOSCBOKU / 2,
            y - DLUGOSCBOKU / 2,
            DLUGOSCBOKU,
            DLUGOSCBOKU) ;
        kwadraty.add (obecnny) ;
        repaint () ;
    }
// Usuwa kwadrat ze zbioru
// @param s kwadrat, który ma zostać usunięty
public void remove (Rectangle2D s) {
    if (s == null) return;
    if (s == obecnny) obecnny = null;
    kwadraty.remove (s) ;
    repaint () ;
}
```

```
private static final int DLUGOSCBOKU = 10;
private ArrayList kwadraty;
private Rectangle2D obecny;
// kwadrat, w którym znajduje się kursor myszki
private class UchwytMyszki extends MouseAdapter{
    @Override
    public void mousePressed(MouseEvent zdarzenie){
// jeśli kursor nie znajduje się wewnątrz kwadratu,
//narysuj nowy kwadrat
        obecny = find(zdarzenie.getPoint());
        if (obecny == null)
            add(zdarzenie.getPoint());
    }
}
```

```
        @Override
        public void mouseClicked(MouseEvent zdarzenie) {
//jeśli nastąpiło podwójne kliknięcie, usuń kwadrat
            obecny = find(zdarzenie.getPoint());
            if (obecny != null &&
                zdarzenie.getClickCount() >= 2)
                remove(obecny);
        }
    }

    private class UchwytRuchuMyszki
        implements MouseMotionListener{
        @Override
        public void mouseMoved(MouseEvent zdarzenie) {
// jeśli kursor myszki znajduje się wewnątrz kwadratu,
// zmień go w krzyżyk
```

```
        if (find(zdarzenie.getPoint()) == null)
            setCursor(Cursor.getDefaultCursor());
        else
            setCursor(Cursor.getPredefinedCursor
                (Cursor.CROSSHAIR_CURSOR));
    }
    @Override
    public void mouseDragged(MouseEvent zdarzenie) {
        if (obecny != null) {
            int x = zdarzenie.getX();
            int y = zdarzenie.getY();
            // przesun' obecny kwadrat tak,
            //aby jego srodek znajdował się w (x, y)
```

```
obecny.setFrame(  
    x - DLUGOSCBOKU / 2,  
    y - DLUGOSCBOKU / 2,  
    DLUGOSCBOKU,  
    DLUGOSCBOKU) ;  
repaint() ;  
}}}}
```

